

Informe de Implementación de Firma Digital

Estudiantes

Carlos Alberto Barriga Gámez
20222020179

Jose Fernando Ramirez Ortiz
20241020080

Joel David Pérez Arroyave
20242020017

Miguel Ángel Hernández Medina
20222020035

Ciencias de la Computación 2

Septiembre de 2026

1. Link repositorio

<https://github.com/ljoeleojl/Talleres-ciencias-2.git>

2. Introducción

Dando continuidad al trabajo anterior, donde se desarrolló un algoritmo de cifrado de una sola vía (*hash*) para representar textos como matrices de colores RGB, en esta entrega se integra un esquema de **Firma Digital** basado en el algoritmo asimétrico RSA.

El objetivo principal es lograr dos propiedades fundamentales en la transmisión de información:

1. **Integridad:** Asegurar que el mensaje no haya sido alterado durante el envío.
2. **Autenticidad:** Confirmar que el mensaje fue firmado únicamente por el emisor que posee la clave privada.

Para cumplir con esto, la matriz RGB obtenida del hash se transforma en un único número entero grande, sobre el cual se aplican las operaciones matemáticas de firma (clave privada) y verificación (clave pública).

3. Desarrollo y Explicación del Código

El sistema se compone de dos clases principales: `CifradoTextoRGB` (utilizada como generador pseudoaleatorio y generadora del hash) y `FirmaSimple`, la cual gestiona la criptografía asimétrica.

3.1. Generación de Claves RSA y Números Primos

Para construir la clave pública y privada, se generan dos números primos grandes de 256 bits mediante la función `generarPrimo`:

```
1 static BigInteger generarPrimo(CifradoTextoRGB generador) {
2     int[] valores = generador.generarEnteros(32, 0, 255);
3     BigInteger candidato = BigInteger.ZERO;
4     for (int valor : valores) {
5         candidato = candidato.multiply(BigInteger.valueOf(256)).
6             add(BigInteger.valueOf(valor));
7     }
8     candidato = candidato.setBit(255); // asegura que el numero
9     return candidato.nextProbablePrime();
10 }
```

El proceso de creación de claves en `generarClaves` realiza los siguientes pasos:

1. Obtiene dos primos de 256 bits, p (`primo1`) y q (`primo2`), mediante el generador Xorshift.
2. Calcula el módulo público $n = p \times q$ (`modulo`) y la función de Euler $\phi(n) = (p - 1)(q - 1)$ (`numeroDeTrabajo`).

3. Se fija el exponente público $e = 65537$ (`clavePublica`) y se calcula el exponente privado d (`clavePrivada`) mediante el inverso modular:

$$d = e^{-1} \pmod{\phi(n)} \quad (1)$$

3.2. Conversión de la Matriz RGB a Número

Dado que el algoritmo RSA trabaja con números enteros, la función `matrizANumero` convierte la matriz de color de $4 \times 4 \times 3$ en un valor numérico continuo procesándolo en base 256:

```
1 static BigInteger matrizANumero(int[][][] matriz) {
2     BigInteger numero = BigInteger.ZERO;
3     for (int[][] fila : matriz) {
4         for (int[] pixel : fila) {
5             for (int valor : pixel) {
6                 numero = numero.multiply(BigInteger.valueOf(256))
7                     .add(BigInteger.valueOf(valor));
8             }
9         }
10    }
11    return numero;
12 }
```

3.3. Proceso de Firma y Verificación

El proceso completo se divide en dos fases matemáticas sencillas:

- **Firma (Emisor):** Eleva el número del hash M al exponente privado d módulo n :

$$\text{Firma} = M^d \pmod{n} \quad (2)$$

```
1 static BigInteger firmar(int[][][] hash) {
2     return matrizANumero(hash).modPow(clavePrivada, modulo);
3 }
```

- **Verificación (Receptor):** Eleva la firma al exponente público e módulo n para recuperar el número original M' , y lo compara con el hash calculado sobre el mensaje recibido:

$$M' = \text{Firma}^e \pmod{n} \quad (3)$$

```
1 static boolean verificar(int[][][] hashRecalculado,
2     BigInteger firma) {
3     BigInteger numeroRecuperado = firma.modPow(clavePublica,
4         modulo);
5     return numeroRecuperado.equals(matrizANumero(
6         hashRecalculado));
7 }
```

4. Resultados y Pruebas

En el método `main` se simula la generación de la firma sobre un mensaje original y su posterior verificación frente a un mensaje alterado.

4.1. Prueba de Validación y Detección de Cambios

Cuadro 1: Resultados del proceso de verificación

Caso de Prueba	Texto Evaluado	Resultado <code>verificar()</code>
Mensaje Original	ciencias de la computacion 2	true
Mensaje Alterado	ciencias de la computacion 3	false

Cuando el receptor evalúa el texto original, el hash reconstruido genera el mismo número que el descifrado a partir de la firma con la clave pública, retornando **true**.

En cambio, cuando el texto cambia mínimamente (cambiando el número 2 por el 3), el efecto avalancha de la clase `CifradoTextoRGB` genera una matriz de color totalmente distinta. Esto produce un número totalmente diferente y la función de verificación detecta la alteración retornando **false**.

5. Conclusiones

- **Garantía de Integridad:** Se cumplió exitosamente el objetivo de asegurar que el mensaje no sea alterado durante la transmisión. Gracias al efecto avalancha del hash a matriz RGB, cualquier modificación por mínima que sea, como cambiar un único carácter, altera por completo la representación numérica del mensaje. Al no coincidir el hash recalculado con el extraído de la firma, el receptor detecta la manipulación inmediatamente.
- **Garantía de Autenticidad :** El sistema confirma que el mensaje fue firmado únicamente por el emisor legítimo. Dado que la firma digital se genera aplicando la clave privada (d), la cual es matemáticamente secreta e intransferible, solo el poseedor de dicha clave pudo haber originado la firma válida que la clave pública (e) logra verificar.
- **Integración Criptográfica Funcional:** La combinación del cifrado de una sola vía (transformación de texto a matriz RGB) con el algoritmo asimétrico RSA demostró que es viable utilizar representaciones visuales/matriciales como insumo para esquemas formales de seguridad digital.
- **Eficiencia en la Generación de Claves:** El uso del generador pseudoaleatorio Xorshift como base para la búsqueda de primos de 256 bits mediante `nextProbablePrime()` resultó ser un mecanismo eficiente y funcional para la construcción del par de claves criptográficas.